

Follow us on Instagram @java_tech_community or visit javatechcommunity.com for the latest Java interview questions.

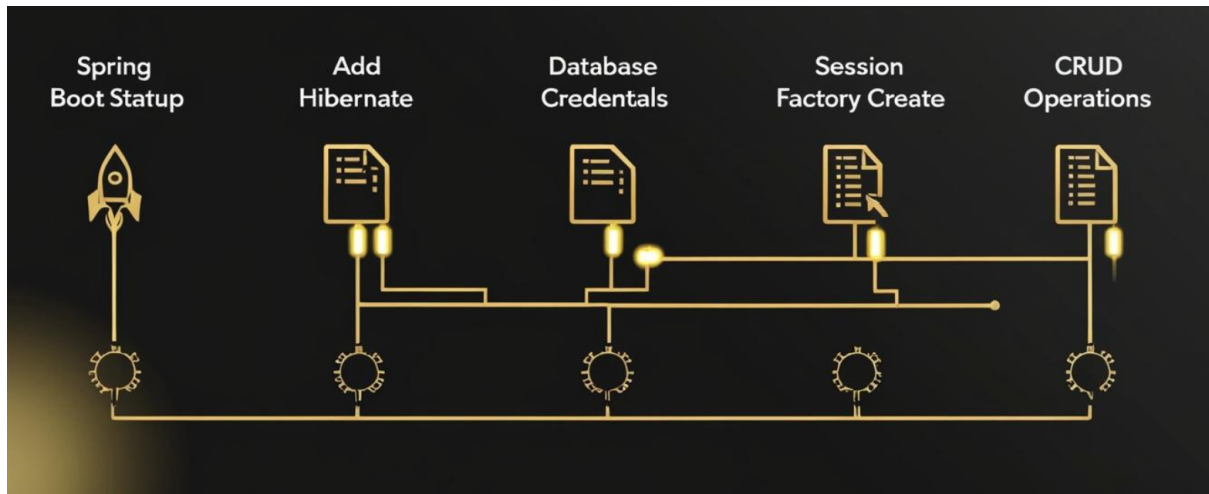
Steps to configure multiple databases with Hibernate in a Spring Boot application:



1. Define the database properties for each database in the application.properties or application.yml file, using different property names for each database.
2. Create two DataSource objects that correspond to the two databases using @Bean annotation.
3. Create two EntityManagerFactory beans that use the corresponding DataSource objects to connect to the databases using @Bean annotation.
4. Use the EntityManager and TransactionManager beans to perform database operations. Autowire the EntityManager and TransactionManager beans in your Spring components to interact with the corresponding databases

Follow us on Instagram @java_tech_community or visit javatechcommunity.com for the latest Java interview questions.

Example Code Configs :



1. Define the database properties for each database in the application.properties or application.yml file.
 - Use different property names for each database.
2. Create two DataSource objects that correspond to the two databases.
 - You can use the @Bean annotation to define each DataSource object.
3. Create two EntityManagerFactory beans that use the corresponding DataSource objects to connect to the databases.
4. Use the EntityManager and TransactionManager beans to perform database operations. You can Autowire the EntityManager and TransactionManager beans in your Spring components to interact with the corresponding databases.

```
//Step 1
spring.datasource.url=jdbc:mysql://localhost:3306/database1
spring.datasource.username=user1
spring.datasource.password=pass1

spring.second-datasource.url=jdbc:mysql://localhost:3306/database2
spring.second-datasource.username=user2
spring.second-datasource.password=pass2

//Step 2
@Bean
@ConfigurationProperties("spring.datasource")
public DataSource dataSource()
{
    ....return DataSourceBuilder.create().build();
}

@Bean(name = "secondDataSource")
@ConfigurationProperties("spring.second-datasource")
public DataSource secondDataSource()
{
    ....return DataSourceBuilder.create().build();
}
```

Follow us on Instagram @java_tech_community or visit javatechcommunity.com for the latest Java interview questions.

Example Code:

```
//Step 3

@Bean
public LocalContainerEntityManagerFactoryBean entityManagerFactory(
    ....EntityManagerFactoryBuilder builder, DataSource dataSource)
{
    ....return builder
    ....    .dataSource(dataSource)
    ....    .packages("com.example.domain")
    ....    .persistenceUnit("default")
    ....    .build();
}

@Bean(name = "secondEntityManagerFactory")
public LocalContainerEntityManagerFactoryBean secondEntityManagerFactory(
    ....EntityManagerFactoryBuilder builder, @Qualifier("secondDataSource") DataSource dataSource)
{
    ....return builder
    ....    .dataSource(dataSource)
    ....    .packages("com.example.domain2")
    ....    .persistenceUnit("second")
    ....    .build();
}

//Step 4

@Autowired
@PersistenceContext(unitName = "default")
private EntityManager entityManager;

@Autowired
@PersistenceContext(unitName = "second")
private EntityManager secondEntityManager;

@Autowired
@Qualifier("transactionManager")
private PlatformTransactionManager transactionManager;

@Autowired
@Qualifier("secondTransactionManager")
private PlatformTransactionManager secondTransactionManager;
```

- In this example, we Autowire the EntityManager and PlatformTransactionManager beans for the default database using the @PersistenceContext and @Qualifier annotations.
- We also Autowire the corresponding beans for the second database.
- These are the basic steps to configure multiple databases with Hibernate in a Spring Boot application. There are additional configuration options available, but these steps should be enough to get you started.